

EchoVault Nexus

Presentation & Live Demo Script

Personal Knowledge & Project Intelligence

EchoVault Nexus: Presentation & Live Demo Script

A step-by-step script for presenting EchoVault Nexus, from sign-in to the end.

Total time: about 20 minutes · **Format:** short introduction, then a live demo

How to read this script:

- **SAY**: the words you speak to the audience (shown in the shaded boxes).
- **DO**: exactly what to click or type.
- **SHOW**: what the audience should notice on screen.

Before you present (preparation checklist)

Do these **10 minutes before** you start:

1. Open a terminal and start the app: `cd ~/Documents/CAAPSTOON-APF/Echovault ./dev.sh` Wait for **"✓ EchoVault Nexus is starting"**. **Keep this terminal open.**
2. Open the browser to **http://localhost:3001** and click around once, so every page is already loaded (the first load of each page is slower).
3. Open a second browser tab at **http://localhost:8025** (the email inbox, used in Part 10).
4. Make sure the AI model is ready: in a terminal, run `ollama list` and check that **qwen2.5:1.5b** appears.
5. Create one note in advance (it is used to show "related notes"): - **Title:** `Nginx rate limit config` - **Content:** `Use limit_req_zone with a burst setting to slow down abusive clients at the proxy.`
6. Zoom the browser to **110–125%** so the audience can read the screen.
7. Close other apps to free memory (the AI runs on your laptop).

Demo account used in this script: Name `Ada Lovelace`, email `ada@example.com`, password `rate limits are fun`. You can use your own details instead.

Part 1: Introduction (about 2 minutes)

Show the landing page at http://localhost:3001 while you talk.

> **SAY:** "Good morning, everyone. Today I'm presenting **EchoVault Nexus**, a personal knowledge and project intelligence platform."
>
> **SAY:** "Let me start with the problem. As developers, students and professionals, we learn things every day, but that knowledge is **scattered**: in chats, bookmarks, random files and our own memory. When we need it, we can't find it."
>
> **SAY:** "A second problem is **lost reasoning**. We remember *that* a decision was made, like 'we use PostgreSQL', but not *why*, what the alternatives were, or whether it worked out. So the same debates happen again and again."
>
> **SAY:** "And when people turn to AI chatbots, the answers come from the internet, not from *their own* knowledge, and there's no way to check where an answer came from."
>
> **SAY:** "EchoVault Nexus solves this with one private, connected workspace. You **capture** notes, **organise** them into projects, **record decisions** with their reasoning, **search** everything instantly, and **ask an AI assistant** that answers only from your own records and shows its sources."

DO: Slowly scroll the landing page.

SHOW: The animated knowledge graph in the hero, the five feature sections, the sample decision record and the **Privacy & security** section.

> **SAY:** "This animation shows the core idea: a note connected to its project, a decision, and references. Now let me show you how it actually works."

Part 2: Creating an account (about 1.5 minutes)

DO: Click **Create account** (top right) or **Create your vault**.

> **SAY:** "First, a new user creates an account."

DO: Fill in the form:

Field	Type this
Full name	`Ada Lovelace`
Email	`ada@example.com`
Password	`rate limits are fun`

DO: Click **Create your vault** *without* ticking the checkbox first.

SHOW: The message *"You need to accept the Terms and Conditions to create an account."*

> **SAY:** "Notice the Terms box starts **unticked**. The user must actively agree, and the app records which version of the Terms they accepted and when. There's no hidden marketing consent."

DO: Tick **I agree to the Terms and Conditions…** and click **Create your vault**.

> **SAY:** "Behind the scenes, the password is never stored as text. It's protected with **Argon2**, a modern password-hashing method. The account is created and I'm signed in securely."

SHOW: The app opens on the **Overview** page.

If the account already exists, use Part 3 instead.

Part 3: Signing in and out (about 1 minute)

DO: Click the round avatar (top right, **AL**) → **Sign out**.

> **SAY:** "Signing out ends the session on the server immediately, not just in the browser."

DO: On the sign-in page, type the email and a **wrong** password, then click **Sign in**.

SHOW: *"Invalid email or password."*

> **SAY:** "The error never reveals whether an email exists, which protects users' privacy. Repeated wrong attempts are also slowed down by rate limiting."

DO: Enter the correct password and click **Sign in**.

> **SAY:** "Sessions use secure cookies that page scripts can't read, and the app quietly renews them while you work."

Part 4: The Overview (about 1 minute)

SHOW: *"Good morning, Ada"*, the counts for **Notes**, **Active projects** and **Decisions**, **Recent activity**, and the **Getting started** steps.

> **SAY:** "This is the Overview, my home screen. These numbers come straight from the database. Nothing is fake. For a new account, it shows three getting-started steps. Let's follow them."

DO: Point at the left sidebar: **Overview, Vault, Projects, Decisions, Assistant, Settings**.

> **SAY:** "The sidebar gives access to every part of the workspace, and the search bar at the top searches everything. I can press the forward-slash key from anywhere to jump to it."

Part 5: Capturing a project (about 2 minutes)

> **SAY:** "Let's say my team is launching a public API. First I create a **project** to hold everything about it."

DO: Click **Projects** in the sidebar → **New project**.

DO: Fill in:

Field	Type this
Name	`Public API launch`
Description	`Ship v1 of the public REST API with keys, docs and rate limits.`
Status	`Active`

DO: Click **Create project**.

SHOW: The project page with its description, status selector, and empty **Knowledge** and **Decisions** sections.

> **SAY:** "A project groups related notes and decisions. It has a status: active, on hold, completed or archived. I can change it right here."

DO: Change the status dropdown to **On hold**, then back to **Active**.

SHOW: The *"Status updated"* message.

> **SAY:** "And if I ever delete a project, its notes and decisions are kept. They're just no longer linked. Nothing is lost by accident."

Part 6: Capturing knowledge in the Vault (about 3 minutes)

> **SAY:** "Now I capture what I learned. On the project page, I click **Add** under Knowledge. The project is already selected for me."

DO: In the **Knowledge** section of the project page, click **Add**.

DO: Fill in the note:

- **Title:** `Token bucket rate limiting`
- **Content:** type the following:

```
## How it works
Each API key gets a **bucket** that refills at a fixed rate.

- Refill: 100 requests per minute
- Burst: 20
- Over the limit: respond with HTTP 429
```

DO: Click the **Preview** tab.

> **SAY:** "Notes are written in Markdown, so I get headings, bold text and lists. I can preview it before saving."

DO: Click **Write** again. In **Tags**, type `api` and press **Enter**, then `performance` and press **Enter**.

> **SAY:** "Tags help me group and filter notes later."

DO: In **Related notes**, type `nginx` and click **Nginx rate limit config**.

> **SAY:** "I can link related notes together. The link works both ways, so the other note will also point back to this one."

SHOW: The **"Unsaved changes"** indicator next to the buttons.

> **SAY:** "The editor shows me when I have unsaved changes, and warns me if I try to leave the page before saving."

DO: Click **Save to vault**.

SHOW: The note page with formatted content, the side panel (Project, Related notes, Tags, dates) and the **Connections** map at the bottom.

> **SAY:** "Here is my saved note. It's stored in a PostgreSQL database, so if I refresh the page or restart the server, it's still here."

DO: Press **F5** to refresh the page.

Part 7: Recording a decision (about 3 minutes)

> **SAY:** "Now the most important feature: the **Decision Journal**. Our team decided how to rate-limit the API. I want to remember not just *what* we chose, but *why*."

DO: Click **Decisions** in the sidebar → **Record decision**.

DO: Fill in:

Field	Type this
Title	`Rate-limit the public API per key`
Status	`Accepted`
Project	`Public API launch`
Context	`A single client can exhaust the database connection pool.`
Problem statement	`Stop abusive bursts without blocking legitimate integrations.`
Alternatives considered	`Per-IP limits at the proxy; scale the database instead.`
Selected option	`Token bucket per API key, 100 requests per minute with bursts of 20.`
Reasoning	`Fair per customer, and needs no extra infrastructure.`
Expected outcome	`No more pool exhaustion; under 1% of requests limited.`

DO: Leave **Actual outcome / review** empty and click **Save decision**.

SHOW: The decision page with the **Selected option** and **Reasoning** highlighted at the top, then context, problem, alternatives, and **Expected vs Actual outcome** side by side.

> **SAY:** "The decision page puts the choice and the reason first. Months later, anyone can see exactly why we did this and what else we considered."
>
> **SAY:** "Decisions also get **reviewed**. Imagine six weeks have passed."

DO: Click **Add review notes**. In **Actual outcome / review**, type: `Zero pool incidents in six weeks; 0.3% of requests limited, mostly one buggy client.` Click **Save changes**.

DO: Click **Mark reviewed**.

SHOW: The status changes to **Reviewed** and a **Last reviewed** date appears.

> **SAY:** "Now we can compare what we expected with what actually happened. That's how a team learns from its decisions."

Part 8: Seeing how everything connects (about 1 minute)

DO: Click **Projects** → **Public API launch**.

SHOW: The project now lists the note and the decision.

DO: Click the note **Token bucket rate limiting** and scroll to **Connections**.

SHOW: The map: the note in the centre, linked to its **Project**, the **Decisions in this project**, and its **Related notes**.

> **SAY:** "This is the 'connected' part of EchoVault. From one note I can see the project it belongs to, the decisions made in that project, and related notes. Every box is a real record I can click."

Part 9: Finding anything with search (about 1.5 minutes)

DO: Press the **/** key, type `limit` and press **Enter**.

SHOW: Results grouped into **Notes** and **Decisions**, with matching words **highlighted**, and filter tabs with counts.

> **SAY:** "Search looks through everything I've written. It matches partial words, so 'limit' also finds 'limiting'. Results are grouped by type and highlighted."

DO: Click the **Decisions** tab, then **All** again.

DO: Search for `perf`.

> **SAY:** "It even matches tags: 'perf' finds notes tagged 'performance'. And search only ever looks at *my* records. Another user can never see them, even with the exact link."

Part 10: Asking the AI assistant (about 2 minutes)

DO: Click **Assistant** in the sidebar.

> **SAY:** "Finally, the AI assistant. Unlike a normal chatbot, it answers **only from my own notes and decisions**, and shows me the sources so I can check the answer."

DO: Type: `What rate limit does our public API use, and why?` and press **Enter**.

> **SAY:** "This model runs locally on my laptop, so my notes never leave my computer. It takes a few seconds."

(While it loads, which can take 15–40 seconds:)

> **SAY:** "Behind the scenes, it first searches my records for the most relevant notes and decisions, wraps them as sources, and sends only those to the AI. Instructions hidden inside a note can't trick it."

SHOW: The answer, plus the **Sources** list with the note and the decision, each clickable.

> **SAY:** "Here's the answer, and below it the exact sources it used. I can click any source to verify it."

DO: Ask: `What is the capital of Mongolia?`

SHOW: *"I couldn't find anything in your notes or decisions that relates to this question..."*

> **SAY:** "And when my notes don't cover a question, it says so honestly, without even contacting the AI. No made-up answers."

Part 11: Account, privacy and settings (about 2 minutes)

DO: Click the avatar → **Settings**.

> **SAY:** "Users stay in control of their data."

SHOW and SAY, one at a time:

1. **Appearance**: click **Light**, then **Dark**. *"Dark, light, or match the system."*
2. **Your data**: click **Download my data**. *"Everything I've stored downloads as a file, so I'm never locked in."*
3. **Legal**: *"It shows which version of the Terms I accepted, and when."*
4. **Security**: *"Sign out everywhere ends my sessions on every device."*
5. **Delete account**: *"I can permanently delete my account and all my data. It asks for my password and for me to type DELETE, so it can't happen by accident."* (Don't actually delete it during the demo.)

Optional: password reset by email

DO: Sign out → **Forgot your password?** → enter the email → **Send reset link**. Switch to the **http://localhost:8025** tab.

SHOW: The reset email in the inbox, with a single-use link.

> **SAY:** "In development, emails are caught by a local test inbox. The reset link works once and expires after 30 minutes."

Part 12: Closing (about 1 minute)

DO: Return to the landing page or the Overview.

> **SAY:** "To summarise: EchoVault Nexus turns scattered knowledge into a **connected, searchable workspace**. Projects keep work organised, the Decision Journal preserves *why* we decided things, search finds anything in seconds, and the AI assistant answers from my own knowledge, with sources I can check."
>
> **SAY:** "It's built with **Next.js** and **React** for the interface, **FastAPI** in Python for the backend, and **PostgreSQL** for the data. It's covered by **over 100 automated tests**, including tests that make sure one user can never see another user's data, and that every page works on screens from 320 to 1440 pixels wide."
>
> **SAY:** "Thank you. I'm happy to take questions."

Likely questions and suggested answers

Question	Suggested answer
Is my data private?	"Yes. Every record belongs to one account, and the server checks ownership on every request. Other users get 'not found' even with the exact link. Passwords are hashed with Argon2."
Does the AI see all my notes?	"No. It only receives the few notes and decisions most relevant to the question, never archived notes, and only when you ask. With the local model, nothing leaves the computer."
What if the AI is wrong?	"Every answer lists its sources with links, so you can check it. If nothing relevant is found, it says so instead of guessing."
Which AI does it use?	"It supports a free local model through Ollama, or Anthropic's Claude with an API key. One setting switches between them."
Why not just use a notes app?	"Notes apps store text. EchoVault connects notes to projects and decisions, keeps the reasoning and outcomes of decisions, and lets you ask questions answered from your own knowledge."
How is it tested?	"69 backend tests, 23 frontend unit tests, and 23 browser tests that click through the real app, all run automatically on GitHub for every change."
Is it live on the internet?	"Not yet. It runs locally. The deployment plan, covering hosting, HTTPS and backups, is documented in the README."
What's next?	"Meaning-based (semantic) search, file uploads, and a full knowledge-graph view."

If something goes wrong during the demo

Problem	Quick fix
The page won't load	Check that the `./dev.sh` terminal is still open and shows no errors; open **http://localhost:3001** exactly.
The assistant is slow	Normal for the free local model (15–40 seconds). Keep talking about how it works while it loads.
"The assistant isn't set up yet"	The AI settings aren't loaded. Restart `./dev.sh`.
A page looks stuck	Refresh with **F5**; your data is saved in the database.
"Port already in use" when starting	An old server is still running. Close the old terminal or press **Ctrl+C** in it, then run `./dev.sh` again.
